Computer Graphics — Rendering

Project 1

Basic Primitives and Display Devices

Purpose

This assignment creates a simple viewing program to display scene description files. You must create device driver routines for the output of PBMplus files, and routines for drawing lines, circles, and flood fills.

Program

You will begin writing a program to process *rendering description*(`rd`) files. RD files are command driven descriptions of a sequence of images. See the documentation for the rd data file format on the course documentation page for details about the commands. Your program will recognize all RD commands, but for now will do very little.

Some support code files are given as the basis for your program along with some additional header files. All of these files can be downloaded from the RD home page, a link to which can be found on the course page for support code. The support code handles the parsing of the RD file commands along with some intelligent error notification. In addition, it contains contains the device drivers for general screen display.

Program Organization

The rendering framework is object oriented and very general. It will theoretically support multiple styles of renderers. For our course, you'll only implement one. Your rendering engine will take the form of a class, named `REDirect` which inherits from the general class `RenderEngine`. Specifications for these classes can be found in `rd_direct.h` and `rd_enginebase.h`. To implement your engine, you will add methods to the `REDirect` class that will take over the default functionality found in the `RenderEngine` base class.

The rendering program is largely under the control of a parser which you do not need to write. As commands are read from the file, they are processed by calling the methods of an instance of a `RenderEngine`. This instance is an instance of your `REDirect` class. These routines are simply an interface between the parser and your graphical processing algorithms.

Your graphical algorithms need a way of plotting a pixel of a given color to the output device. These basic routines are provided for you in `rd_display.h`. These routines are device independent. Your graphical routine should not know nor care what device a pixel is being directed to. The connection between these basic plotting routines and a specific device is made by the parser when a `Display` command is read.

The routines in `rd_display` will call the appropriate device driver routines. If the selected device is the screen, routines provided in the library will be used. You must provide the routines used for rendering off-screen images that will be saved as a PPM file. These device driver routines that you should implement are found in `pnm_display.h`.

In summary, the program `main()` (provided) calls the parser (provided) which reads commands from a file and calls either routines in `rd_direct.h`(you implement) or default routines in `rd_enginebase.h`(provided). Your

interface and rendering routines call the generic pixel plotting routines (provided) in `rd_display.h` which call, depending on the chosen device, screen display routines (provided) or PPM display routines (you_implement).

When finished, your program will be able to handle the `rd` commands `Display`, using both `Screen` and `PNM` devices, `Format`, `FrameBegin` and `FrameEnd`, `WorldBegin` and `WorldEnd`, `Background`, `Color`, `Point`, `Line`, `Circle`, and `Fill`.

Important Operation Note: When displaying images on the screen, the program is set up to terminate when the image is clicked with the right mouse button. This is the best way to end the program.

As an additional aid, when displaying images on the screen, clicking the image with the left mouse button will give information about the pixel underneath the tip of the cursor.

Implementation

Following the above description, there are two groups of functions you must implement: those called through `rd_enginebase.h` and those found in `pnm_display.h`.

You must add *all* of the methods listed below to your `REDirect` class. The names and arguments should mirror those found in the `RenderEngine` base class. Initially, implement them as stubs. All of the routines should return an error code. The most common return value will be `RD_OK`. Other possible error code values can be found in `rd_error.h`. After creating functioning stubs, implement the following routines more fully:

`rd_display()`
: Actually, the major work for this function is done behind the scenes. Nothing else need be done here.

`rd_format()`
: Ditto. The values of the x and y resolution are stored automatically in the global variables `display_xSize` and `display_ySize` found in `rd_display.h`, but if you want to store them somewhere else as well, this is the time to do it.

`rd_frame_begin()`
: Store the frame number in a global variable somewhere.

`rd_world_begin()`
: The key thing here is to initialize the display for a new image. This does not mean a completely new display setup. It may be just a matter of clearing things out for a new frame. The `rd_disp_init_frame()` function found in `rd_display.h` is specifically designed for this. Eventually, you will add much more to this routine.

`rd_world_end()`
: Finish off the display frame here. See `rd_display.h`

`rd_frame_end()`
: Same as `rd_world_end()` at this point. But don't just have one call the other. They will change.

`rd_color`
: Store the r, g, and b values in some global location as the current drawing color. This will be the basic color of every primitive drawn until it is changed. Store these values as floating point numbers. The default drawing color should be white.

`rd_background`
: Store the color values to be used as the screen background color. The `rd_set_background()` routine in `rd_display.h` should be used here. The default background color should be black.

`rd_point`
: Write a single pixel using the current drawing color.

`rd_line`
: Should call your line drawing routine and draw a line using the current drawing color.

`rd_circle`
: Calls your circle drawing routine and draws a circle using the current drawing color.

`rd_fill`

Calls your flood fill routine using the given seed point and the current drawing color as the fill color.

You must also implement the following driver routines (see `pnm_display.h`) for sending the display to a PPM file:

`pnm_init_display()`
: Where everything for initially setting up a device is done. The global values `display_xSize` and `display_ySize` in `rd_display.h` are guaranteed to have valid values at this point. All that needs to be done here is dynamically allocate some global memory sufficiently large to hold the image of the given size.

`pnm_end_display()`
: Called when the input file is finished. Free up the dynamically allocated image memory here.

`pnm_init_frame()`
: Get ready for a new image. Just clear out the image array. Save the frame number into a global location.

`pnm_end_frame()`
: Surprisingly, this is the most involved routine. When this is called, the image is finished and can be written out to storage. The file name is found in the global variable `display_name` which must be modified by the frame number in any way you see fit to create a unique file name. Appending works well. Output a standard PPM file from the global image data.

`pnm_write_pixel()`
: Set the image value at location (x, y) to the value of the current color. Don't forget any necessary intensity range conversion.

`pnm_read_pixel()`
: Read the value from location(x, y) in the array to the red, green, and blue values passed in. Make sure that the output values are in the range 0.0 to 1.0.

`pnm_set_background()`
: Store the color values in some global location as the current background color. The default background color should be black.

`pnm_clear()`
: (Re)initialize the image array to the background color.

NOTE: Some simple scene files for testing your renderer can be found on the graphical data page.

Input and Output

By using or modifying the sample scene files on the course web page for data, or by creating your own scene files, perform the following tasks.

The scene files you create/use here should be turned in as part of your assignment.

Lines

1. Draw a set of lines to the screen using single buffering. Make sure that all basic orientation categories are represented.
2. Draw the same set of lines to the screen using double buffering.
3. Draw the same set of lines to a file.

Circles

1. In the left half of a new image, draw a set of concentric circles in various colors with radii of odd lengths, in the range of 1 - 11.
2. In the right half of the same image, draw a set of concentric circles with radii of even lengths from 2 - 12.

Flood Fill

1. Draw a square. Draw a horizontal line and a vertical line through the square. Replace the pixel where the lines cross with the background color. (The interiors of the quadrants are now 8-connected, but not 4-connected). Fill each quadrant with a different color.
2. Draw a set of concentric circles and fill them to make a bullseye pattern with colors of brown, dark gray, light gray, pink, red, orange, yellow, green, blue and purple in order from the center circle out. Don't use the automatic default colors (i.e. red - R = 1.0, G = 0.0, B = 0.0). Experiment and find some pleasing colors that would be used in a child's paint set. Your colors are not expected to match those of other people in the class.

Your Picture Here

Design a scene using your new tools. This is an important part of your grade, typically 10% of the grade for this assignment.

Submission

Submit all of your source code files and scene description files (rd files) using the method described on the course assignment page. Don't submit images unless they are fabulous errors and will never be reproduced again in your functioning render engine.

Last modified: Thu Feb 8 10:42:22 CST 2024